I've prepared a consolidated summary that you can use as a project reference or presentation notes.

Dynamic Evolutionary Optimization Algorithm Switching for AutoML

Final Year Project Summary

1. Project Overview

This project proposes a **Dynamic Evolutionary Optimization Framework** for **Automated Hyperparameter Optimization (AutoML)**. Instead of relying on a single optimization algorithm throughout the search process, the framework continuously monitors the performance of multiple evolutionary algorithms and dynamically reallocates computational resources to the best-performing algorithms.

The framework is **model-independent** and **dataset-independent**, allowing it to optimize hyperparameters for any machine learning model through a black-box objective function.

---

2. Problem Statement

Existing hyperparameter optimization methods rely on a single fixed optimization algorithm, which may stagnate or become inefficient during different stages of the search, leading to wasted computational resources. This project proposes a dynamic evolutionary optimization framework that adaptively switches between algorithms to improve search efficiency and convergence.

---

3. Motivation

Hyperparameter optimization is computationally expensive because every candidate solution requires training and evaluating a machine learning model.

Traditional optimization methods such as:

- Grid Search
- Random Search
- Genetic Algorithm
- Particle Swarm Optimization
- Differential Evolution

continue using the same optimization strategy even when it stops making meaningful progress.

Different optimization algorithms perform better at different stages of the search. Therefore, an adaptive framework can utilize computational resources more efficiently.

---

4. Objective

Design a framework that:

- Optimizes hyperparameters automatically.
- Dynamically switches between optimization algorithms.
- Detects algorithm stagnation.
- Promotes high-performing algorithms.
- Improves convergence and sample efficiency.
- Remains independent of any machine learning model or dataset.

---

5. Core Components

Configuration Space

Defines all searchable hyperparameters and their valid ranges.

Example:

- Learning Rate
- Batch Size
- Optimizer
- Number of Layers

It contains **only search boundaries**, not datasets or models.

Objective Function (Fitness Evaluator)

Responsible for:

- Loading the dataset
- Creating the ML model
- Training the model
- Evaluating performance
- Returning the fitness score

Example output:

Accuracy = 94.2%

The optimizer never knows how the model is trained.

This makes the optimization process a **black-box optimization problem**.

Dynamic Evolutionary Optimizer

The optimizer:

- Generates candidate hyperparameters.
- Sends them to the objective function.
- Receives fitness scores.
- Learns from previous evaluations.
- Dynamically selects the best optimization algorithm.

---

6. Black-Box Optimization

The optimizer only knows:

```
Hyperparameters
        ↓
Objective Function
        ↓
Fitness Score
```

It never interacts directly with:

- datasets
- CNNs
- XGBoost
- Transformers
- loss functions

Therefore, the framework is reusable for any machine learning model.

---

7. Overall Workflow

```
User

↓
```

```
Define Configuration Space

↓

Write Objective Function

↓

Start Optimization

↓

Tier 1 Algorithms

↓

Generate Hyperparameter Candidates

↓

Objective Function

↓

Train Model

↓

Return Fitness

↓

Composite Scoring Engine

↓

UCB1 Decision

↓

Promotion / Demotion

↓

Tier 2 Elite Algorithms

↓

Repeat Until Budget Ends

↓
```

```
Return Best Hyperparameters
```

---

8. Outer Loop vs Inner Loop

Outer Loop

The optimization process.

Each iteration:

- algorithms generate candidate vectors
- evaluation occurs
- algorithm performance is updated
- switching decisions are made

Inner Loop

Each candidate vector requires:

- loading dataset
- training model
- validating model
- returning fitness

This is the computationally expensive part.

---

9. Why AutoML Is Expensive

Every candidate solution requires a complete model evaluation.

Example:

1000 evaluations

×

3 minutes per model

=

3000 minutes

≈ 50 hours

The optimizer should therefore minimize wasted evaluations.

---

10. Sample Efficiency

Sample efficiency refers to finding better hyperparameters using fewer model evaluations.

Your framework improves sample efficiency by:

- avoiding stagnated algorithms
- reallocating resources
- sharing elite solutions
- dynamically switching algorithms

---

11. Composite Scoring Engine

The objective function evaluates **hyperparameter configurations**.

The Composite Scoring Engine evaluates **optimization algorithms**.

It computes five normalized metrics:

1. Best Fitness

Highest accuracy discovered by the algorithm.

---

2. Population Diversity

Measures how different candidate solutions are.

High diversity:

- better exploration

Low diversity:

- possible stagnation

---

3. Performance Trend

Measures improvement over recent iterations.

Positive slope:

Algorithm is improving.

Flat slope:

Algorithm has stagnated.

---

4. Convergence Speed

Measures how quickly an algorithm improves.

Faster improvement receives a higher score.

---

5. Computational Efficiency

Measures the optimizer's internal computation time.

It excludes machine learning training time.

---

12. Dynamic Switching

Initially multiple algorithms search simultaneously.

Example:

- GA
- PSO
- DE
- ABC

After several iterations:

The Composite Scoring Engine ranks them.

The UCB1 Multi-Armed Bandit:

- promotes strong algorithms
- reduces resources for weak algorithms
- detects stagnation
- reallocates computational budget

---

13. Tier 1 and Tier 2

Tier 1

All algorithms compete equally.

Tier 2

Elite algorithms receive:

- larger populations
- more computational resources
- cooperative information sharing

Algorithms can also be demoted back to Tier 1.

---

14. Cooperative Co-evolution

Elite algorithms exchange their best hyperparameter configurations.

Example:

GA discovers:

Learning Rate = 0.01

PSO discovers:

Batch Size = 64

Both algorithms benefit from each other's discoveries.

This accelerates convergence.

---

15. Convergence

Convergence is the process by which an optimization algorithm gradually approaches the optimal hyperparameter solution through successive iterations.

Example:

Iteration	Accuracy
1	80%

Iteration	Accuracy
5	88%
10	92%
15	94%
20	94.2%
25	94.2%

Fast convergence:

Finds good solutions quickly.

Premature convergence:

Stops improving because it is trapped in a local optimum.

---

16. Stopping Criterion

The default stopping criterion is the **evaluation budget**.

Example:

Budget = 1000 evaluations.

The optimizer continues until all evaluations are consumed.

Early stopping can optionally be added using:

- target accuracy
- no improvement for N iterations
- time limit
- convergence threshold

For research benchmarking, a fixed evaluation budget ensures fair comparison among optimization algorithms.

---

17. Use Case

The framework automatically tunes hyperparameters for any machine learning model.

Example applications:

- CNN image classification
- XGBoost
- Random Forest
- Regression
- Time-series forecasting
- Tabular classification

Only the objective function changes.

The optimization engine remains unchanged.

---

18. Advantages

Flexibility

The optimizer is independent of datasets and machine learning models.

It works with any objective function.

Dynamic Adaptation

Algorithms are continuously monitored and switched based on performance.

Better Sample Efficiency

Fewer wasted model evaluations.

Resource Allocation

More computational effort is assigned to better-performing algorithms.

Faster Convergence

Elite cooperation and dynamic switching accelerate optimization.

---

19. Research Contribution

Instead of assuming one optimization algorithm is always the best, the proposed framework continuously evaluates multiple algorithms and dynamically adapts the optimization strategy throughout the search process.

---

20. One-Line Project Description

A dynamic AutoML hyperparameter optimization framework that adaptively switches between multiple evolutionary algorithms using performance-based evaluation and a UCB1 Multi-Armed Bandit to improve convergence, sample efficiency, and computational resource utilization while remaining independent of the underlying machine learning model and dataset.